

Тема: Функції та об'єкти PHP**План лекції**

1. Функції, визначені користувачем.
2. Аргументи функцій.
3. Списки аргументів змінної довжини.
4. Використання змінних всередині функції.
5. Значення, що повертаються.

Зміст лекції

У цій лекції будуть розглядатися питання створення і використання функцій в PHP. Говорячи «функції», ми не маємо на увазі всі існуючі в PHP функції, а лише функції, визначені користувачем. Ми розглянемо способи завдання таких функцій, методи передачі аргументів, використання аргументів зі значенням за замовчуванням і значення, що повертаються функцією.

1. Функції, визначені користувачем.

Навіщо потрібні функції, спробуємо пояснити на прикладі. Класичний приклад функції в програмуванні - це функція, що обчислює значення факторіала числа. Тобто ми задаємо їй число, а вона повертає нам його факторіал. При цьому не потрібно для кожного числа, факторіал якого ми хочемо отримати, повторювати один і той же код - досить просто викликати функцію з аргументом, рівним цьому числу.

Функція обчислення факторіала натурального числа

```
<?php
function fact($n) {
    if ($n == 0) return 1;
    else return $fact = $n * fact($n-1);
}
echo fact(3);
    // можна було б написати echo (3 * 2);
    // але якщо число велике,
echo fact(50);
    // то зручніше користуватися функцією,
    // ніж писати echo (50 * 49 * 48 * ... * 3 * 2);
?>
```

Таким чином, коли ми здійснюємо дії, в яких простежується залежність від якихось даних, і при цьому, можливо, нам знадобиться виконувати такі ж дії, але з іншими вихідними даними, зручно використовувати механізм функцій - оформити блок дій у вигляді тіла функції, а змінні дані - як її параметрів.

Подивимося, як в загальному вигляді виглядає завдання (оголошення) функції. Функція може бути визначена за допомогою наступного синтаксису:

```
function ім'я_функції(параметр1, параметр2, ... параметрN) {
    блок_дій
    return «значення, яке повертає функція»;
}
```

Якщо прямо так написати в php-програмі, то працювати нічого не буде. По перше, **ім'я_функції** і імена параметрів функції (**параметр1**, **параметр2** і т.д.) повинні відповідати правилам найменування в PHP (кириличні символи в них краще не використовувати). Імена функцій нечутливі до регістру. По-друге, параметри функції - це змінні мови, тому

перед назвою кожного з них повинен стояти знак \$. Ніяких крапок ставити в списку параметрів не можна. По-третє, замість слів **блок дій** в тілі функції повинен знаходитися будь-який правильний PHP-код (не обов'язково залежить від параметрів). І нарешті, після ключового слова `return` має йти коректний php-вираз (що-небудь, що має значення). Крім того, у функції може і не бути параметрів, як і значення, що повертається. Приклад правильного оголошення функції - функція обчислення факторіала, наведена вище.

Як відбувається виклик функції? Вказується ім'я функції і в круглих дужках список значень її параметрів, якщо такі є:

```
<?php
```

```
Ім'я_функції («значення_параметру1», «значення_параметру2», ...);
```

```
// приклад виклику функції - виклик функції обчислення факторіала наведено вище,
```

```
// там для обчислення факторіала числа 3 ми писали: fact (3);
```

```
// де fact - ім'я функції, що викликається, а 3 - значення її параметра з ім'ям $n
```

```
?>
```

Коли можна викликати функцію? Здавалося б, дивне запитання. Функцію можна викликати після її визначення, тобто в будь-якому рядку програми. Єдиний виняток становлять функції, які визначаються умовно (всередині умовних операторів або інших функцій). Коли функція визначається таким чином, її визначення повинно передувати її виклику.

```
<?php
```

```
$make = true;
```

```
/* Тут не можна викликати Make_event (); тому що вона ще не існує, але можна  
викликати Save_info() */
```

```
Save_info( "Вася", "Іванов", "Я вибрав курс по PHP");
```

```
if ($make) {
```

```
    // визначення функції Make_event ()
```

```
    function Make_event () {
```

```
        echo "<p> Хочу вивчати Python <br>";
```

```
    }
```

```
}
```

```
// тепер можна викликати Make_event ()
```

```
Make_event ();
```

```
// визначення функції Save_info
```

```
function Save_info($first, $last, $message) {
```

```
    echo "<br> $message <br>";
```

```
    echo "Ім'я:". $First. ". $Last. "<br>";
```

```
}
```

```
Save_info ( "Федя", "Федоров", "А я вибрав Lisp");
```

```
// Save_info можна викликати і тут
```

```
?>
```

Приклад. Визначення функції усередині умовного оператора

2. Аргументи функцій.

У кожній функції може бути, як ми вже говорили, список аргументів. За допомогою цих аргументів у функцію передається різна інформація (наприклад, значення числа, факторіал якого треба підрахувати). Кожен аргумент є змінною або константою.

За допомогою аргументів дані в функцію можна передавати трьома різними способами. Це передача аргументів за значенням (використовується за умовчанням), по

посиланню і завдання значення аргументів за замовчуванням. Розглянемо ці способи докладніше.

Коли аргумент передається в функцію за значенням, зміна значення аргументу всередині функції не впливає на його значення поза функцією. Щоб дозволити функції змінювати її аргументи, їх потрібно передавати за посиланням. Для цього у визначенні функції перед ім'ям аргументу слід написати знак амперсанд «&».

```
<?php
// напишемо функцію, яка б додавала до рядка слово checked
function add_label(&$data_str) {
    $data_str.= " Checked";
}
$str = "<input type = radio name = article";
// нехай є такий рядок
echo $str. "> <br>";
// виведе елемент форми - не відмічену радіо кнопку
add_label($str);
// викличемо функцію
echo $str. "> <br>";
// це виведе вже відмічену радіо кнопку
?>
```

Приклад. Передача аргументів за посиланням

У функції можна визначати значення аргументів, використовуваних за замовчуванням. Саме значення за замовчуванням має бути константним виразом, а не змінною і не представником класу або викликом іншої функції.

У нас є функція, що створює інформаційне повідомлення, підпис до якого змінюється в залежності від значення переданого їй параметра. Якщо значення параметра не задано, то використовується підпис "Оргкомітет".

```
<?php
function Message ($sign = "Оргкомітет.") {
    // тут параметр sign має за замовчуванням значення "Оргкомітет"
    echo "Наступні збори відбудуться завтра. <br>";
    echo $sign. "<br>";
}
Message ();
// викликаємо функцію без параметра. У цьому випадку підпис - це Оргкомітет
Message ("З повагою, Вася.");
// У цьому випадку підпис буде "З повагою, Вася."
?>
```

Приклад. Значення аргументів за замовчуванням

Результатом роботи цього скрипта буде:

Наступні збори відбудуться завтра.

Оргкомітет.

Наступні збори відбудуться завтра.

З повагою, Вася.

Якщо у функції кілька параметрів, то ті аргументи, для яких задаються значення за замовчуванням, повинні бути записані після всіх інших аргументів у визначенні функції. В іншому випадку з'явиться помилка, якщо ці аргументи будуть опущені при виклику функції.

Наприклад, ми хочемо внести опис статті в каталог. Користувач повинен ввести такі характеристики статті, як її назва, автора та короткий опис. Якщо користувач не вводить ім'я автора статті, вважаємо, що це Іванов Іван.

```
<?php
function Add_article ($title, $description, $author = "Іванов Іван") {
    echo "Заносимо в каталог статтю: \"\$title\", ";
    echo "автор $author";
    echo "<br> Короткий опис: ";
    echo "$description <br>";
}
Add_article ( "Інформатика і ми", "Це стаття про інформатику ...", "Петров
Петро");
Add_article ( "Хто такі хакери", "Це стаття про хакерів ...");
?>
```

В результаті роботи скрипта одержимо наступне
Заносимо в каталог статтю: Інформатика і ми, автор Петров Петро.
Короткий опис: Це стаття про інформатику ...

Заносимо в каталог статтю: Хто такі хакери, автор Іванов Іван.
Короткий опис: Це стаття про хакерів ...
Якщо ж ми напишемо ось так:

```
<?php
function Add_article($author = "Іванов Іван", $title, $description) {
    // ... дії як в попередньому прикладі
}
Add_article ( "Хто такі хакери", "Це стаття про хакерів ...");
?>
```

То в результаті отримаємо:

 Expected 3 arguments. Found 2.intelephense(P1005) [9, 13]

3. Списки аргументів змінної довжини.

Починаючи з PHP4 можна створювати функції зі змінним числом аргументів. Тобто ми створюємо функцію, не знаючи заздалегідь, з якою кількістю аргументами її викличуть. Для написання такої функції ніякого спеціального синтаксису не потрібно. Все робиться за допомогою вбудованих функцій `func_num_args()`, `func_get_arg()`, `func_get_args()`.

Функція `func_num_args()` повертає число аргументів, переданих в поточну функцію. Ця функція може використовуватися тільки усередині визначення користувальницької функції. Якщо вона з'явиться поза функцією, то інтерпретатор видає попередження.

```
<?php
function DataCheck() {
    $n = func_num_args();
    echo "Число аргументів функції $n";
}
DataCheck();
    // виведе рядок
    // «Число аргументів функції 0»
DataCheck(1,2,3);
    // виведе рядок
    // «Число аргументів функції 3»
?>
```

Приклад. Використання функції func_num_args ()

Функція func_get_arg (цілий номер_аргументу) Повертає аргумент зі списку переданих у функцію аргументів, порядковий номер якого заданий параметром номер_аргументу. Аргументи функції рахуються починаючи з нуля. Як і func_num_args (), ця функція може використовуватися тільки усередині визначення будь-якої функції.

Номер_аргументу не може перевищувати число аргументів, переданих у функцію. Інакше буде згенеровано попередження, і функція func_get_arg () поверне False.

Створимо функцію для перевірки типу даних її аргументів. Вважаємо, що перевірка пройшла успішно, якщо перший аргумент функції - ціле число, другий - рядок.

```
<?php
function DataCheck () {
    $Check = true;
    $n = func_num_args ();
    // число аргументів, переданих у функцію
    /* Перевіряємо, чи є перший переданий аргумент цілим числом */
    if ($n >= 1) {
        if (!is_int (func_get_arg (0)))
        {
            $Check = false;
        }
    }
    /* Перевіряємо, чи є другий переданий аргумент рядком */
    if ($n >= 2) {
        if (!is_string (func_get_arg (1)))
        {
            $Check = false;
        }
    }
    return $Check;
}

if (DataCheck("123", "text"))
    {echo "Перевірка пройшла успішно <br>";}
else {echo "Дані не задовольняють умовам <br>";}
if (DataCheck(324, "22")) {echo "Перевірка пройшла успішно <br>";}
else {echo "Дані не задовольняють умовам <br>";}

?>
```

Приклад. Функція для перевірки типу даних її аргументів

Результатом роботи буде наступне.

Перевірка пройшла успішно

Перевірка пройшла успішно

Функція func_get_args () повертає масив, що складається зі списку аргументів, переданих функції. Кожен елемент масиву відповідає аргументу, переданому функції. Якщо функція використовується поза визначення користувальницької функції, то генерується попередження.

Перепишемо попередній приклад, використовуючи цю функцію. Будемо перевіряти, чи є цілим числом кожен парний аргумент, який передається функції:

```
<?php
function DataCheck() {
    $Check = true;
    $n = func_num_args(); //число аргументів, переданих у функцію
```

```

$args = func_get_args(); //масив аргументів функції
for ($i = 1; $i<$n+1; $i++) {
    $v = $args[$i-1];
    if ($i % 2 == 0) {
        if (!is_int($v)) {
            $Check = false; //перевіряємо, чи є парний аргумент цілим
        }
        else $Check = true;
    }
}
return $Check;
}
if (DataCheck("text", 22))
echo "Перевірка пройшла успішно <br>";
else
echo "Дані не задовольняють умовами <br>";
?>

```

Як бачимо, комбінації функцій `func_num_args()`, `func_get_arg()` і `func_get_args()` використовується для того, щоб функції могли мати змінний список аргументів.

4. Використання змінних всередині функції.

Глобальні змінні

Щоб використовувати всередині функції змінні, задані поза нею, ці змінні потрібно оголосити як глобальні. Для цього в тілі функції слід перерахувати їх імена після ключового слова `global`, наприклад: `global $var1, $var2;`

```

<?php
$a = 1;
function Test_g() {
    global $a;
    $a = $a * 2; echo 'в результаті роботи функції $a =', $a;
}
echo 'поза функцією $a =', $a, ', ';
Test_g();
echo "<br>";
echo 'поза функцією $a =', $a, ', ';
Test_g();
?>

```

Приклад. Глобальні змінні

В результаті роботи цього скрипта одержимо:

поза функцією `$a = 1`, в результаті роботи функції `$a = 2`

поза функцією `$a = 2`, в результаті роботи функції `$a = 4`

Коли змінна оголошується як глобальна, фактично створюється посилання на глобальну змінну. Тому такий запис еквівалентна наступній (масив `$GLOBALS` містить всі змінні, глобальні щодо поточної області видимості):

```
$var1 = &$GLOBALS["var1"];
```

```
$var2 = &$GLOBALS["var2"];
```

Це означає, наприклад, що видалення змінної `$var1` не видаляє глобальної змінної `$GLOBALS["var1"]`.

Статичні змінні

Щоб використовувати змінні тільки всередині функції, при цьому зберігаючи їх значення і після виходу з функції, потрібно оголосити ці змінні як статичні. Статичні

змінні видно тільки всередині функції та вони не втрачають свого значення, якщо виконання програми виходить за межі функції. Оголошення таких змінних проводиться за допомогою ключового слова `static`:

```
static $var1, $var2;
```

статичної змінної може бути присвоєно будь-яке значення, але не посилання.

```
<?php
function Test_s() {
    static $a = 1;
    // не можна привласнювати вираз або посилання
    $a = $a * 2;
    echo $a;
}
Test_s(); // виведе 2
echo $a; // виведе "Undefined variable", так як $a доступна тільки всередині
функції
Test_s(); // всередині функції $a = 2, тому результатом роботи функції буде число
4
?>
```

Приклад. Використання статичної змінної

5. Значення, що повертаються.

Всі функції, наведені вище як приклади, виконували будь-які дії. Крім подібних дій, будь-яка функція може повертати як результат своєї роботи якесь значення. Це робиться за допомогою оператора **return**. Значення, що повертається може бути будь-якого типу, включаючи списки і об'єкти. Коли інтерпретатор зустрічає команду `return` в тілі функції, він негайно припиняє виконання, цієї функції, і переходить на той рядок, з якої була викликана функція.

Наприклад, складемо функцію, яка повертає вік людини. Якщо людина не померла, то вік вважається відносно поточного року.

```
<?php
/* Якщо другий параметр обчислюється як true, то він розглядається як дата
смерті*/

function Age($birth, $is_dead) {
    if ($is_dead) return $is_dead- $birth;
    else return date ( "Y" ) - $birth;
}
echo Age (1971, false); // для 2009 року виведе 38
echo Age (1971, 2001); // виведе 30
?>
```

У цьому прикладі можна було і не використовувати функцію `return`, а просто замінити її функцією виведення **echo**. Однак якщо ми все ж робимо так, що функція повертає якесь значення (в даному випадку вік людини), то в програмі ми можемо присвоїти будь-якій змінній значення цієї функції:

```
$an_age = Age (1981, 2004);
```

В результаті роботи функції може бути повернуто тільки одне значення. Кілька значень можна отримати, якщо повертати список значень (одновимірний масив). Припустимо, ми хочемо отримати повний вік людини з точністю до дня.

```
<?php
function Full_age ($b_day, $b_month, $b_year)
```

```

{
    $y = date("y");
    $m = intval(date ( "m"));
    $d = intval (date ( "d"));
    $b_month = intval ($b_month);
    $b_day = intval ($b_day);
    $b_year = intval ($b_year);

    $Day = ($b_day > $d? 30 - $b_day + $d: $d - $b_day);
    $TmpMonth = ($b_day > $d? -1: 0);
    $Month = ($b_month > $m + $TmpMonth? 12 - $b_month +
        $TmpMonth + $m: $m + $TmpMonth - $b_month);
    $TmpYear = ($b_month > $m + $TmpMonth? -1: 0);
    if ($b_year > $y + $tmpYear)
    {
        $Year = 0; $Month = 0; $Day = 0;
    }
    else
    {
        $Year = $y + $tmpYear - $b_year;
    }
    return array ($Day, $Month, $year);
}
$age = Full_age ( "29", "06", "1986");
echo "Вам $age[2] років, $age[1] місяців і $age[0] днів";
?>

```

Коли функція повертає кілька значень для їх обробки в програмі, зручно використовувати мовну конструкцію `list ()`, яка дозволяє однією дією привласнити значення відразу декільком змінним. Наприклад, в попередньому прикладі, залишивши без зміни функцію, обробити значення які повертаються функцією, можна було так:

```

<?php
// завдання функції Full_age()
list ($Day, $Month, $year) = Full_age( "07", "08", "1974");
echo "Вам $year років, $Month місяців і $Day днів";
?>

```

Взагалі конструкцію `list ()` можна використовувати для присвоєння змінним значень елементів будь-якого масиву.

```

<?
$arr = array ("first", "second");
list ($a, $b) = $arr;
    // змінної $a присвоюється перше значення масиву, $b - друга
echo $a, "", $b;
    // виведе рядок "first second"
?>

```

Приклад. Використання `list ()`

Повернення посилання

В результаті своєї роботи функція також може повертати посилання на будь-яку змінну. Це може стати в нагоді, якщо потрібно використовувати функцію для того, щоб визначити, який змінній повинно бути присвоєно посилання. Щоб отримати з функції посилання, потрібно при оголошенні перед її ім'ям написати знак амперсанд (`&`) і кожен раз при виконанні функції перед її ім'ям теж писати амперсанд (`&`). Зазвичай функція

повертає посилання на будь-яку глобальну змінну (або її частина - посилання на елемент глобального масиву), посилання на статичну змінну (або її частина) або посилання на один з аргументів, якщо він був також переданий по посиланню.

```
<?php
$a = 3; $b = 2;
function &ref($par) {
    global $a, $b;
    if ($par% 2 == 0) return $b;
    else return $a;
}
$var = &ref(4);
echo $var, " i ", $b, "<br>";
    // виведе 2 i 2
$b = 10;
echo $var, " i ", $b, "<br>";
    // виведе 10 i 10
?>
```

Приклад. повернення посилання

При використанні синтаксису посилань в змінну **\$var** нашого прикладу не копіюється значення змінної **\$b** повернутої функцією **\$ref**, А створюється посилання на цю змінну. Тобто тепер змінні **\$var** і **\$b** ідентичні і змінюватимуться одночасно.

Контрольні питання.

1. **Користувачькі функції:** Як створити власну функцію в PHP? Опишіть синтаксис `function name() { ... }` та механізм повернення значень через `return`.
2. **Аргументи функції:** Як передати аргументи у функцію? Чи можна передавати аргументи за посиланням (`&$var`) і як це впливає на оригінальну змінну?
3. **Область видимості:** Що таке область видимості змінних? Як отримати доступ до глобальної змінної всередині функції за допомогою ключового слова `global`?
4. **Статичні змінні:** Для чого використовуються статичні змінні (`static`) всередині функції? Як вони зберігають своє значення між викликами функції?
5. **Функції зі змінною кількістю аргументів:** Як працюють функції `func_num_args()`, `func_get_arg()` та `func_get_args()` для обробки довільної кількості переданих параметрів?
6. **Глобальний масив `$_GET`:** Як отримати дані, передані методом GET? У яких випадках цей метод передачі даних є доречним?
7. **Глобальний масив `$_POST`:** Як отримати дані з форми, відправленої методом POST? Чому цей метод безпечніший для передачі паролів?
8. **Обробка форм:** Опишіть типовий алгоритм обробки HTML-форми на PHP: перевірка натискання кнопки submit, отримання даних, валідація, виконання дій.
9. **Масив `$GLOBALS`:** Що містить суперглобальний масив `$GLOBALS`? Як через нього можна звертатися до глобальних змінних?
10. **Повернення кількох значень:** Як функція може повернути декілька значень одночасно? Опишіть використання конструкції `list()` при отриманні результату у вигляді масиву.
11. **Рекурсія:** Чи підтримує PHP рекурсивні виклики функцій? Наведіть приклад завдання, яке вирішується рекурсивно (наприклад, факторіал).
12. **Валідація типів:** Як перевірити тип переданого аргументу всередині функції (наприклад, `is_int()`, `is_string()`)?

